Diseño y Arquitectura de Software

Módulo 3 – Desarrollo Guiado por Pruebas TDD

Facultad de
Ingeniería

VIGILADA MINEDUCACIÓN

Universidad de La Sabana

Universidad de La Sabana | Facultad de **Ingeniería**



Bienvenidos

En este módulo, nos adentraremos en el mundo de **TDD**, esta técnica revoluciona la forma en que se escribe y se prueba el código, enfocándose en la creación de pruebas antes de implementar la funcionalidad. En esta presentación, exploraremos los beneficios de TDD, el ciclo de desarrollo que lo caracteriza y cómo aplicarlo de manera efectiva en proyectos de software.



¿Qué el Desarrollo Guiado por Pruebas?

El Desarrollo Guiado por Pruebas (TDD, por sus siglas en inglés Test-Driven Development) es una metodología de desarrollo de software que se basa en escribir pruebas antes de escribir el código de producción.

Ciclo de Desarrollo

El ciclo de TDD consta de tres pasos:

1. Escribir una prueba que falle.
2. Escribir el código mínimo necesario para que la prueba pase.
3. Refactorizar el código para mejorarlo sin cambiar su funcionalidad.

Filosofía

TDD se fundamenta en la idea de que el código debe ser diseñado en función de las necesidades del usuario y las pruebas que se desean realizar sobre él. Esto ayuda a mantener un código más limpio, modular y fácilmente mantenible.



Facultad de Ingeniería

Universidad de La Sabana

El Ciclo de Desarrollo TDD

1

Escribir la Prueba

El primer paso del ciclo TDD es escribir una prueba unitaria que falle. Esto obliga al desarrollador a pensar en la funcionalidad deseada antes de escribir el código de producción.

2

Escribir el Código

Una vez que se ha escrito la prueba, el siguiente paso es implementar la funcionalidad mínima necesaria para que la prueba pase. El objetivo es escribir el código más simple posible para cumplir con los requisitos.

3

Refactorizar

Después de que la prueba pase, el código se refactoriza para mejorar su diseño, legibilidad y mantenibilidad, sin cambiar su funcionalidad. Las pruebas unitarias garantizan que no se introduzcan nuevos bugs durante el proceso de refactorización.

Ventajas de TDD:

Calidad de Código

Al escribir las pruebas primero, los desarrolladores se ven obligados a pensar detenidamente en la interfaz y la funcionalidad de su código. Esto resulta en un código más modular, flexible y fácilmente mantenible.

Reducción de Bugs

TDD ayuda a identificar y corregir errores de manera temprana en el proceso de desarrollo. Al tener una suite de pruebas automatizada, los desarrolladores pueden refactorizar el código con confianza y con la seguridad de que no se introducirán nuevos bugs.

Desarrollo Ágil

TDD se alinea perfectamente con los principios ágiles, permitiendo a los equipos iterar rápidamente, entregar valor de manera continua y responder mejor a los cambios. Las pruebas automatizadas proporcionan un "colchón de seguridad" que facilita el desarrollo evolutivo.



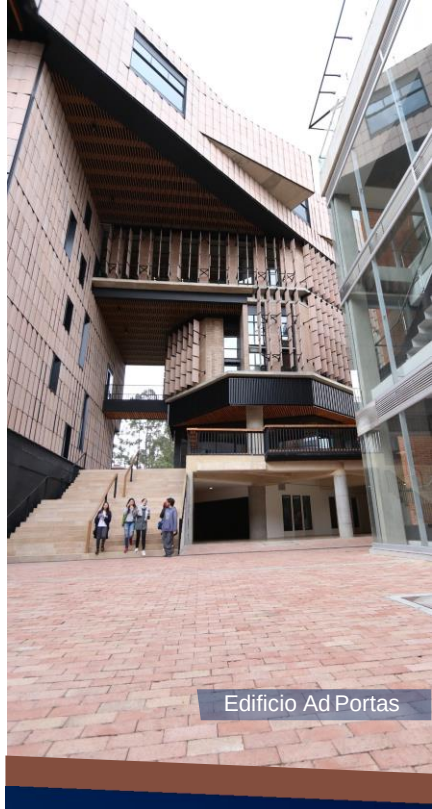
Pruebas Unitarias

¿Qué son?

Las pruebas unitarias son pruebas automatizadas que verifican el correcto funcionamiento de una unidad de código, como una función o un método, de forma aislada. Estas pruebas se enfocan en comprobar la lógica interna y el comportamiento de pequeñas porciones de código.

Principios

Las pruebas unitarias de calidad deben ser rápidas, independientes, repetibles y enfocadas en una única responsabilidad. Además, deben ser fáciles de escribir, mantener y entender.



Facultad de
Ingeniería

Universidad de
La Sabana

Pruebas Unitarias

Ejemplos

Algunos ejemplos de pruebas unitarias incluyen verificar que una función devuelve el resultado esperado, que un método lanza la excepción correcta o que una clase realiza una operación específica de manera adecuada.

Beneficios

Las pruebas unitarias permiten detectar y corregir errores rápidamente, facilitan la refactorización, mejoran la documentación del código y promueven un diseño más modular y desacoplado.



Facultad de
Ingeniería

Universidad de
La Sabana

Cobertura de Código

¿Qué es?

La cobertura de código es una métrica que mide qué porción del código fuente de una aplicación está siendo ejercitada por las pruebas automatizadas. Es una herramienta útil para evaluar la calidad y la efectividad de las pruebas.

Métricas

Las principales métricas de cobertura de código son la cobertura de líneas, la cobertura de ramas y la cobertura de funciones. Estas métricas proporcionan información sobre qué partes del código están siendo probadas y cuáles podrían necesitar más atención.

Cuidado con la Falacia

Si bien la cobertura de código es una métrica útil, no debe ser el único indicador de calidad. Una alta cobertura no garantiza necesariamente que las pruebas sean de alta calidad o que no existan bugs. Es importante enfocarse también en la efectividad y la relevancia de las pruebas.





Cobertura de instrucciones (Statement Coverage)

Esta métrica evalúa la cantidad de instrucciones del código fuente que han sido ejecutadas al menos una vez durante la ejecución de las pruebas. Es la métrica de cobertura más básica y fácil de entender. Sin embargo, puede no ser suficiente para garantizar que todas las ramas del código hayan sido probadas.

Cobertura de instrucciones (Statement Coverage):

$$\text{Statement Coverage } (\%) = \left(\frac{\text{Número de instrucciones ejecutadas}}{\text{Número total de instrucciones}} \right) \times 100$$

Cobertura de ramas (Branch Coverage)

Esta métrica evalúa la cantidad de ramas de control de flujo que han sido ejecutadas al menos una vez durante la ejecución de las pruebas. Las ramas incluyen declaraciones condicionales, bucles y estructuras de control similares. Proporciona una visión más detallada de qué partes del código han sido probadas y puede ayudar a identificar áreas no cubiertas por pruebas.

Cobertura de ramas (Branch Coverage):

$$\text{Branch Coverage } (\%) = \left(\frac{\text{Número de ramas ejecutadas}}{\text{Número total de ramas}} \right) \times 100$$

Cobertura de condición (Condition Coverage)

Esta métrica es similar a la cobertura de ramas, pero se centra específicamente en las condiciones booleanas dentro de las estructuras de control, como las condiciones de los bucles y las declaraciones if. Evalúa si todas las posibles condiciones han sido evaluadas tanto a verdadero como a falso durante las pruebas.

Cobertura de condición (Condition Coverage):

$$\text{Condition Coverage } (\%) = \left(\frac{\text{Número de condiciones verdaderas ejecutadas} + \text{Número de condiciones falsas ejecutadas}}{\text{Número total de condiciones}}\right) \times 100$$

Cobertura de función (Function Coverage)

Esta métrica evalúa si todas las funciones o métodos definidos en el código han sido llamados al menos una vez durante las pruebas. Es útil para identificar funciones que no se utilizan en absoluto en el código y que podrían ser eliminadas o revisadas.

Cobertura de función (Function Coverage):

$$\text{Function Coverage } (\%) = \left(\frac{\text{Número de funciones ejecutadas}}{\text{Número total de funciones}}\right) \times 100$$



Cobertura de línea (Line Coverage)

Esta métrica es similar a la cobertura de instrucciones, pero se enfoca en las líneas individuales de código en lugar de en las instrucciones. Puede proporcionar una visión más detallada de qué líneas de código han sido ejecutadas durante las pruebas.

Cobertura de ruta (Path Coverage)

Esta métrica evalúa si todas las posibles rutas a través del código han sido ejecutadas al menos una vez durante las pruebas. Es la métrica más exhaustiva y compleja, ya que implica probar todas las combinaciones posibles de ramas de control de flujo.

Cobertura de línea (Line Coverage):

$$\text{Line Coverage } (\%) = \left(\frac{\text{Número de líneas ejecutadas}}{\text{Número total de líneas}} \right) \times 100$$





Calidad de las Pruebas Unitarias

Claridad

Las pruebas deben ser claras, concisas y fáciles de entender, para facilitar la lectura y el mantenimiento.

Repetibilidad

Las pruebas deben ser repetibles y producir los mismos resultados cada vez que se ejecuten.

Velocidad

Las pruebas deben ejecutarse rápidamente, para permitir una retroalimentación ágil y frecuente.

Enfoque

Cada prueba debe tener un objetivo claro y una única responsabilidad, sin probar múltiples funcionalidades.

Patrones de Pruebas Unitarias

Arrange-Act-Assert

Este patrón divide una prueba unitaria en tres etapas:

1) Preparar los datos de entrada
2) Ejecutar la funcionalidad a probar
3) Verificar que se obtenga el resultado esperado.

Given-When-Then

Este patrón sigue una estructura narrativa para expresar las pruebas:

1) Dado un contexto específico
2) Cuando se ejecuta una acción
3) Entonces se espera un resultado determinado.

Spy/Mock

Estos patrones permiten aislar una unidad de código de sus dependencias, reemplazándolas por objetos ficticios que simulan su comportamiento. Esto facilita el testeo y la verificación de interacciones.



Facultad de Ingeniería

Universidad de La Sabana

Métricas de Calidad

Entender las métricas clave utilizadas en las pruebas de software es fundamental para evaluar el éxito y la calidad de un proyecto. Estas métricas proporcionan una visión clara del desempeño del proceso de pruebas y ayudan a tomar decisiones informadas.



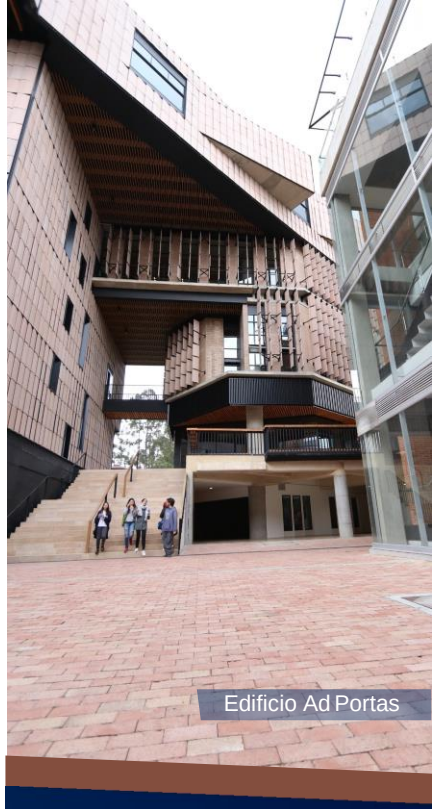
Tasa de Defectos

¿Qué es?

La tasa de defectos mide la relación entre el número de defectos encontrados y el número total de casos de prueba ejecutados. Esta métrica permite evaluar la efectividad del proceso de pruebas y detectar áreas que necesitan mejoras.

Ejemplo

Sí encontraste 10 defectos y ejecutaste 100 casos de prueba, la tasa de defectos sería 10/100 = 0.1 o 10%.



Facultad de
Ingeniería

Universidad de
La Sabana

Tiempo Promedio de Reparación

¿Qué es?

El tiempo promedio de reparación (MTTR) indica el tiempo promedio que toma reparar los defectos detectados. Esta métrica ayuda a estimar los recursos y el esfuerzo necesarios para mantener la calidad del software a lo largo del tiempo.

Ejemplo

Sí el tiempo total de reparación de 5 defectos es de 30 horas, entonces el MTTR sería 30 horas / 5 defectos = 6 horas por defecto.



Facultad de
Ingeniería

Universidad de
La Sabana

Tiempo Promedio entre Fallas

¿Qué es?

El tiempo promedio entre fallas (MTBF) mide el tiempo promedio entre la aparición de defectos. Esta métrica proporciona información sobre la confiabilidad del sistema y ayuda a planificar mejor las actividades de mantenimiento y pruebas.

Ejemplo

Sí encontraste 4 defectos en un período de 20 días, el MTBF sería 20 días / 4 defectos = 5 días.



Facultad de Ingeniería

Universidad de La Sabana

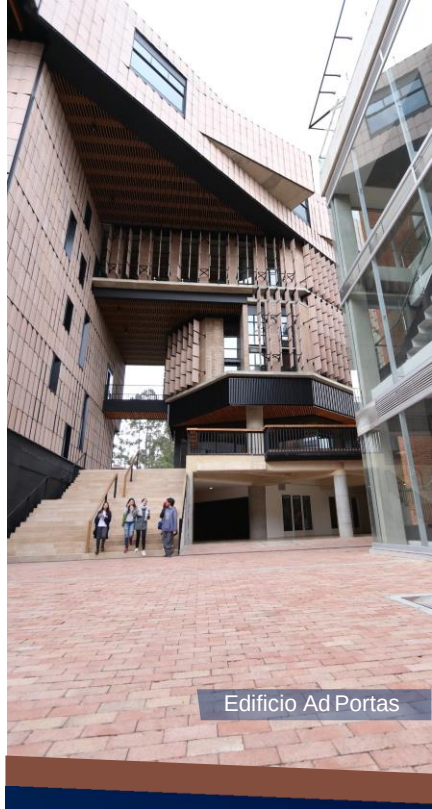
Cobertura de Requisitos

¿Qué es?

La cobertura de requisitos evalúa qué porcentaje de los requisitos del sistema están siendo probados. Esta métrica permite identificar áreas del sistema que no están siendo suficientemente validadas y priorizar pruebas adicionales.

Ejemplo

Divide el número de requisitos cubiertos por las pruebas por el número total de requisitos en el sistema.
Multiplica el resultado por 100 para obtener el porcentaje.



Facultad de
Ingeniería

Universidad de
La Sabana



Aplicando las Métricas

Análisis de Tendencias

Monitorear el comportamiento de las métricas a lo largo del tiempo permite identificar patrones y áreas de mejora continua.

Establecer Objetivos

Definir metas específicas para cada métrica ayuda a enfocar los esfuerzos de pruebas y mejorar la calidad del software.

Toma de Decisiones

Las métricas de pruebas proporcionan información valiosa para tomar decisiones informadas sobre el desarrollo y la liberación del software.

Aplicando las Métricas

1 Mejora Continua

Las métricas permiten identificar y corregir problemas de calidad de manera más efectiva.

2 Optimización de Recursos

La información proporcionada por las métricas ayuda a asignar los recursos de manera más eficiente.

3 Mayor Confianza

Las métricas de pruebas aumentan la confianza en la calidad y el desempeño del software.

4 Toma de Decisiones Informada

Las métricas permiten tomar decisiones más fundamentadas durante todo el ciclo de vida del software.



Pruebas de Integración

1

¿Qué son?

Las pruebas de integración se enfocan en verificar que los diferentes componentes de un sistema funcionan correctamente cuando se combinan. Estas pruebas validan la interacción entre las unidades de código y la correcta integración de los diferentes módulos.

2

Tipos de Pruebas

Existen diferentes tipos de pruebas de integración, como las pruebas de pila, pruebas de subsistemas, pruebas de caja negra y pruebas de usuario final. Cada tipo de prueba tiene un enfoque y una complejidad diferente.

3

¿Cuándo usarlas?

Las pruebas de integración son esenciales cuando se trabaja con sistemas complejos, que involucran múltiples componentes y dependencias. Estas pruebas ayudan a detectar problemas de interoperabilidad y aseguran que el sistema funcione correctamente como un todo.



Facultad de Ingeniería

Universidad de La Sabana



Estrategias de Pruebas de Integración

Top-Down

Las pruebas de arriba hacia abajo comienzan con los módulos de alto nivel y prueban su interacción con módulos más simples.

Bottom-Up

Las pruebas de abajo hacia arriba inician con los módulos más básicos y avanzan hacia los de mayor complejidad.

Sandwich

La estrategia sandwich combina las pruebas de arriba hacia abajo y de abajo hacia arriba.

Técnicas de Pruebas de Integración

1 Big Bang

Prueba todos los componentes integrados de una sola vez.

2 Stubs y Drivers

Simulan la interacción con componentes externos para probar la integración.

3 Escenarios de Integración

Identifican los casos de uso clave para probar la integración.

4 Configuración de Pruebas

Preparar el entorno de pruebas con datos y escenarios realistas.





Facultad de Ingeniería

Universidad de La Sabana

Diseño de Pruebas de Integración

1

Identificar Componentes

Determinar qué módulos y subsistemas se deben integrar.

2

Definir Escenarios

Establecer casos de uso relevantes para probar la integración.

3

Configurar Entorno

Preparar el ambiente de pruebas con datos y condiciones realistas.

Ejecución de Pruebas de Integración

1

Preparar Entorno

Configurar el ambiente de pruebas con los componentes a integrar.

2

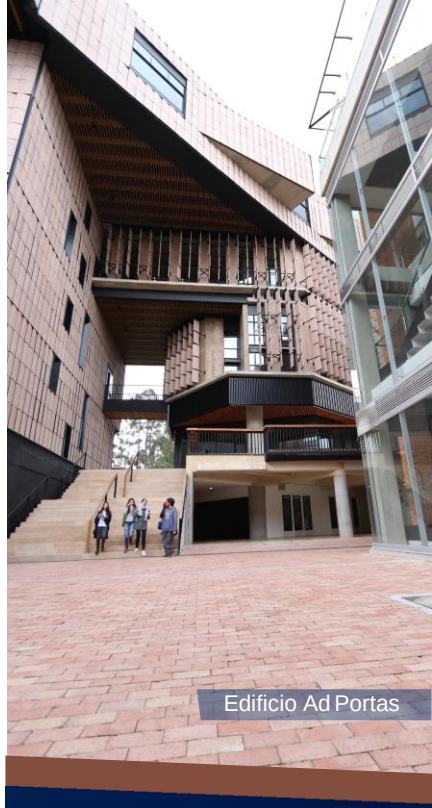
Ejecutar Pruebas

Probar los escenarios de integración diseñados previamente.

3

Analizar Resultados

Registrar y evaluar los resultados de las pruebas de integración.



Facultad de
Ingeniería

Universidad de
La Sabana



Herramientas para Pruebas de Integración

Postman

Herramienta popular para pruebas de APIs.

TestNG

Framework de pruebas para Java.

NUnit

Framework de pruebas para .NET.

Beneficios de las Pruebas de Integración

Detección de Errores

Identificar problemas de interoperabilidad entre componentes.

Garantía de Calidad

Asegurar el correcto funcionamiento del sistema integrado.

Optimización de Costos

Reducir esfuerzos de depuración en etapas posteriores.

Confianza en el Producto

Entregar un software robusto y confiable a los usuarios.



Clases de equivalencia

1

¿Qué son?

Las clases de equivalencia son conjuntos de valores de entrada que se espera que se procesen de la misma manera por el sistema.

2

Objetivo

Identificar los casos de prueba más representativos, reduciendo el número de pruebas necesarias sin perder cobertura.

3

Beneficios

Aumenta la eficiencia y efectividad de las pruebas, al centrarse en los casos más relevantes.



Facultad de Ingeniería

Universidad de La Sabana



Identificación de Clases de Equivalencia

Análisis de Requisitos

Comprender el comportamiento esperado del sistema y las restricciones de los datos de entrada.

Partición de Entrada

Dividir el dominio de entrada en grupos de datos similares que se espera que se procesen de la misma manera.

Selección de Casos de Prueba

Elegir un caso de prueba representativo de cada clase de equivalencia para validar el sistema.

Tipos de Clases de Equivalencia

1 **Válidas**

Representan entradas que el sistema puede procesar correctamente.

2 **No Válidas**

Representan entradas que el sistema no puede procesar o que deben generar errores.

3 **Fronterizas**

Representan valores límite entre clases válidas y no válidas.



Ejemplo de Clases de Equivalencia

1 Entrada de Edad

Clase válida: 18 a 65 años. Clase no válida: menores de 18 y mayores de 65 años.

2 Entrada de Saldo

Clase válida: 0 a 10.000 $. Clase no válida: saldo negativo.

3 Entrada de Correo Electrónico

Clase válida: correo electrónico con formato válido. Clase no válida: correo electrónico con formato inválido.



Implementación de Clases de Equivalencia

1

Diseño de Pruebas

Definir los casos de prueba que cubren las clases de equivalencia identificadas.

2

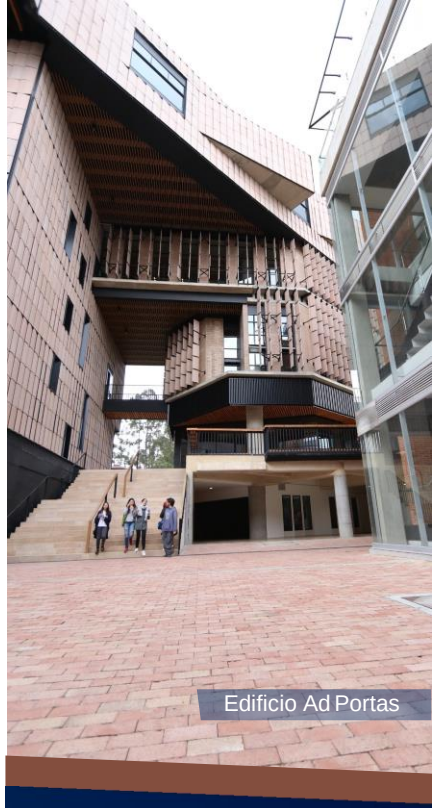
Ejecutar Pruebas

Ejecutar los casos de prueba y evaluar los resultados obtenidos.

3

Corrección de Defectos

Identificar y corregir los defectos encontrados durante las pruebas.



Facultad de
Ingeniería

Universidad de
La Sabana



Ventajas de las Clases de Equivalencia

Eficiencia

Reducción del número de casos de prueba necesarios.

Efectividad

Mayor cobertura de pruebas y detección de defectos.

Ahorro de Tiempo

Optimización del proceso de pruebas.

Facultad de
Ingeniería



Universidad de
La Sabana